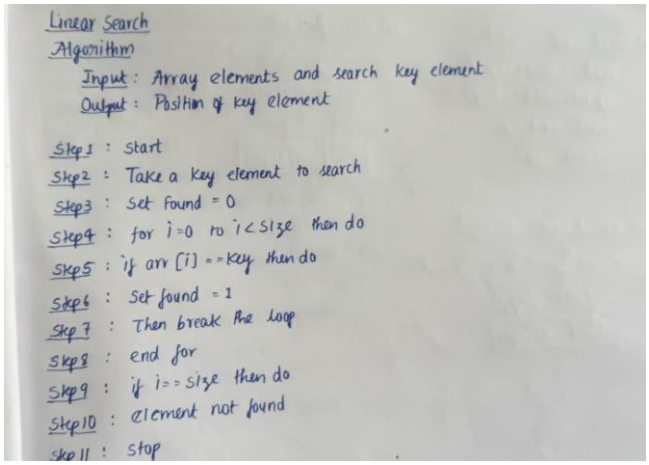
PRACTICAL-02

AIM: Implementation and Time analysis of linear and binary search algorithm.

1.) LINEAR SEARCH

ALGORITHM:


Linear Search
Algorithm
Input : Array elements and search key element
Output : Position of key element

Step 1 : Start
Step 2 : Take a key element to search
Step 3 : Set found = 0
Step 4 : for i=0 to i<size then do
Step 5 : if arr[i] == key then do
Step 6 : Set found = 1
Step 7 : Then break the loop
Step 8 : end for
Step 9 : if i == size then do
Step 10 : element not found
Step 11 : stop

PROGRAM:

```
#include <iostream>

using namespace std;

int main() {

   int arr[20],n,key,i;

   cout << "Enter the number of elements: ";

   cin >> n;

   cout << "Enter the array elements: ";

   for (int i = 0; i < n; i++) {

      cin >> arr[i];

   }

   cout << "Enter the element to search: ";

   cn >> key;

   for (int i = 0; i < n; i++) {

      if (arr[i] == key) {

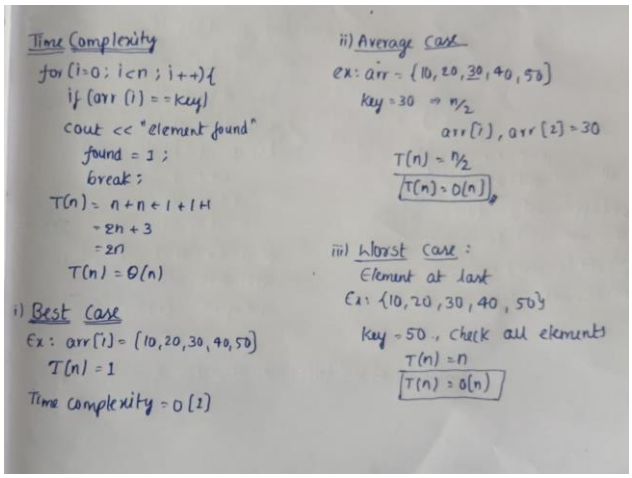
         cout << "Element found at index " << i;
```

```
        return 0;
      }
   }
   cout << "Element not found";
   return 1;
}
```

OUTPUT:

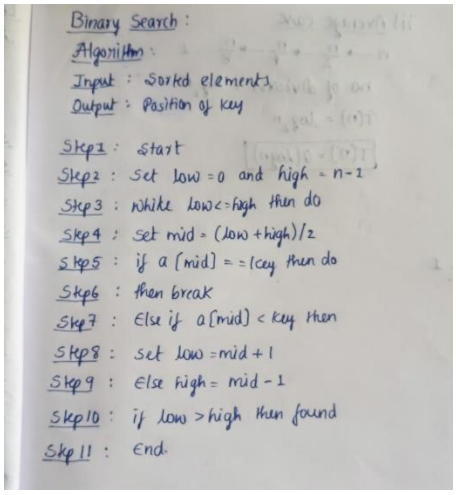
```
Enter the number of elements: 5
Enter the array elements: 10 20 30 40 50
Enter the element to search: 30
Element found at index 2
```

TIME COMPLEXITY:



2.) BINARY SEARCH

ALGORITHM:


Binary Search :
Algorithm :
Input : Sorted elements
Output : Position of key
Step1 : start
Step2 : Set low = 0 and high = n-1
Step3 : while low<=high then do
Step4 : set mid = (low + high)/2
Step5 : if a[mid] == key then do
Step6 : then break
Step7 : Else if a[mid] < key then
Step8 : set low = mid + 1
Step9 : Else high = mid - 1
Step10 : if low > high then found
Step11 : End.

PROGRAM:

```
#include <iostream>
using namespace std;
int main()
{
   int n, key;
   int low, high, mid;

   cout << "Enter the number of elements: ";
   cin >> n;
   int a[n];
   cout << "Enter the sorted array elements: ";
   for (int i = 0; i < n; i++)
   {
      cin >> a[i];
   }
   cout << "Enter the key element: ";
   cin >> key;
   low = 0;
   high = n - 1;
   while (low <= high)
   {
      mid = (low + high) / 2;
      if (a[mid] == key)
      {
```

```
            cout << "Element found at position " << mid << endl;
            break;
        }

        else if (a[mid] < key)
        {
            low = mid + 1;
        }

        else
        {
            high = mid - 1;
        }
    }
    if (low > high)
    {
        cout << "Element not found" << endl;
    }
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter the sorted array elements: 10 20 30 40 50
Enter the key element: 30
Element found at position 2
```

TIME COMPLEXITY:

Time Complexity:

```
low = 0 ; high = n - 1 ;        - 1
while (low <= high) {           - logn
    mid = (low + high)/2 ;      - logn
  if (a[mid] == key)            - logn
        break ;                 - 1
    else if (a[mid] < key)      - logn
        low = mid - 1 ;         - logn
    else
        high = mid - 1;         - logn
    }
```

$T(n) = 1 + \log n + \log n + \log n + 1 + 1 + \log n + \log n + \log n$

$= 2 + 6 \log n$

$T(n) = \theta(\log n)$

Because in every iteration, array divides into half

